1 Comparing Distinct-Count Sketches Under Equal Memory

Format: Experimental, supported by short theoretical derivations.
Members: Tinlone Cheng (z5692826), Siddhant Jain (z5692832), Matthias Lei (z5692836), Aaron Li (z5688334), Daniel Zhang (z5693586)
Mentor: Liam Heng

1.1 Topic and Objective

In large databases with many duplicate elements or keys, it is infeasible to store and compare every individual element to find the number of unique elements. As such, we may require algorithms to perform *cardinality estimation (CE)*, which allows us to count the number of unique users, generate usage analytics, distribute resources appropriately and optimise query times.

Our audience is COMP3821 students who know hashing and basic probability but have not learnt about streaming sketches.

Many techniques exist for estimating cardinality with varying accuracy and variance of their estimates, parallelisability, running time, memory consumption and scalability. In particular, we will focus on probabilistic data structures, and analyse how, under equivalent memory budgets, bitmap-based, filter-based, order-statistic based and register-based algorithms differ in estimation accuracy and mergeability across different cardinality regimes, as well as how well their theoretical error guarantees predict the observed errors.

Our objective is to determine which algorithm is 'best' (lowest bias and RMSE) under a particular set of circumstances by analytically and empirically analysing them.

1.2 Initial Background and Understanding

Cardinality estimation (CE), commonly referred to as the Count-Distinct Problem (CDP), is the problem of estimating the number of distinct elements in a multiset or data stream without having to examine every element. Instead, we use probabilistic sketches that trade a small amount of accuracy for large gains in speed and memory efficiency.

The canonical application of CDP is counting the unique visitors of a high traffic website or the unique IP addresses in a large network. Another primary use case is query optimisation in databases. Query planners rely on cardinality estimates to efficiently choose join orders, select access methods, and allocate resources. Poor estimates will result in significantly slower queries, so accurate and efficient CE is critical.

Several characteristics should be considered when choosing a CE algorithm. Accuracy is the most important, as the estimate should have low bias and variance, typically measured by relative error. Next, in terms of speed, updates and queries need to be sufficiently fast for streaming data. Memory efficiency is important as the sketch must operate efficiently under the memory budgets it is designed for. Scalability matters too, for the algorithm must handle high volume/bandwidth streams of distinct elements. Finally, mergeability allows multiple sketches be efficiently merged without spending extra resources reprocessing.

We will use the original paper for each family of algorithms as our primary source. Whang et al. introduced Linear Counting, an estimator which reads cardinality directly from the fraction of empty bits in a bitmap. If n distinct items are hashed uniformly into m bits, a given bit stays empty with probability $(1 - 1/m)^n \approx e^{-n/m}$, so if V is the observed fraction of empty bits, $\hat{n} = -m \ln V$; the estimate fails once the bitmap saturates ($V = 0$). It serves as our baseline because its bias and variance can be derived from first principles, making it a reliable reference for judging more complex estimators.

Similarly, Flajolet and Martin's probabilistic counting scheme also uses a bitmap, but introduces stochastic averaging, using many small bitmaps, on which later register-based methods are build on.

Not every sketch employs a register-based method. Bar-Yossef et al. proposed the k-minimum-values estimator, which encodes stream information fundamentally differently. It retains an ordered sample of the smallest hash values rather than using a fixed-size occupancy pattern. This lets us test whether the accuracy of our cardinality estimate depends on the sketch's representation or on the number of bits it is allocated.

The register-based path then develops into the LogLog family. It begins with Durand and Flajolet's original LogLog (LL) and continues with Flajolet et al.'s HyperLogLog (HLL). HLL replaces LL's geometric mean with a harmonic mean and adjusts the error constant. Heule et al.'s HyperLogLog++ adds bias correction and sparse representation, making the method reliable at small cardinalities.

1.3 Format and Intended Outcomes

We will complete an experimental project by implementing our stream generator and algorithms in C++, then using Python scripts to orchestrate the pipeline and collect/analyse the results. Each algorithm will be evaluated on the same series of tests, primarily comparing their ability to estimate the number of distinct elements in a large data stream without storing every item. We will implement the algorithms identified, namely linear counting, k-minimum values, bloom filters and HyperLogLog (HLL). If time permits, we will also implement extensions of the LogLog family, in particular HLL++ and UltraLogLog. Across various lengths of data streams, we will test or analyse:

- accuracy of the estimate, measured by bias, RMSE and 95% confidence intervals for fixed memory budgets $B \in \{\,1,\,4,\,8,\,16\,\}$ KiB.
- observed RMSE of each sketch against the predicted theoretical guarantee and its variance
- mergeability of two disjoint partitions of a stream

The calculation of expectations, probabilities, and 'simple' distributions such Binomial and Poisson will be introduced, as well as the definitions of (sample) mean and variance for use in derivations or mathematical proofs. Where proofs or definitions are far beyond the scope of the course, such as the Beta distribution in KMV or Mellin Transform in the HLL proof, we will quote results instead of deriving them.

Results and derivations will be compiled in the report and poster, and compare the different approaches, examining the assumptions they rely on, how they work, mathematical guarantees and derivations, as well as how they trade off space, accuracy and mergeability.

1.4 Proposed Approaches and Evaluations

We propose to compare and analyse the effectiveness of each CE algorithm in different circumstances by deriving theoretical predictions for the expected value and error bounds of each algorithm to predict their accuracy. We will then implement each algorithm with the same hash function with their original papers as the main source and to design experiments to both compare the theoretical and experimental characteristics of each algorithm, and to compare the algorithms against each other.

As the mathematical proof for HLL is beyond the scope of this project, we will instead use the published error formula from Flajolet et al. and explain the intuition behind it.

Our current experimental approach is to test the algorithms using the four memory budgets from above, varying the number of unique items from very small counts to very large streams, with multiple trials and different hash seeds, from which we can calculate the root-mean square error and average systematic error (bias). For each measurement, we will run $T = 64$ trials, and calculate the mean and sample standard deviation, the standard errors of which reduce as $\frac{1}{\sqrt{T}}$.

Additionally, we will attempt to source some SQL databases to test our algorithms on. If this turns out to be too difficult (whether that be because the datasets are too large to process locally, or our SQL ability turns out to be insufficient to allow us to pass it in), we will fall back to dynamically generating data in Python.

To compare the sketches fairly, we give each the same memory budget B, counted in bits of sketch

state, which is the data the algorithm stores plus any extra information it needs to maintain its estimate, but excluding the hash function, the analysis code and the input stream. We choose the largest bitmap, set of HLL registers or KMV sample that fits within each limit, and report the storage actually used. For instance, Linear Counting and the Bloom filter use a bitmap of up to B bits, KMV stores up to $B/64$ hash values. Every sketch uses the same seeded 64-bit hash function. Our test streams are random 64-bit identifiers with duplicates chosen uniformly at random, and we repeat the runs with a skewed distribution.

We treat a theoretical prediction as *confirmed* when it falls inside the 95% interval of the measured RMSE, within the range of n where that estimator's formula applies. Where it does not, we investigate and explain the gap.

We check each implementation against an exact hash-set count on small streams (KMV must be exact for $n < k$), confirm that repeats and reordering leave the sketch state unchanged, confirm that a merged sketch has the same state as one built directly from the union, and compare estimates with a reference implementation. Over many seeds the mean error should match each estimator's predicted bias.

We also plan to empirically analyse other characteristics of these algorithms, such as the mergeability. I.e., whether and how well the algorithm can merge two disjoint partitions of a stream with memory budget B and cardinalities d_1 and d_2 into a single sketch with memory budget B and cardinality $d_1 + d_2$, and how accurate this is.

1.5 Plan

1.5.1 Project Stages, Milestones and Timeline

Week	Actions	Milestones
3 (Stage I)	Set up repo. Assign roles.	Finalise proposal.
4	Define algorithms to research. Implement a data generator and choose a hash function to use, and linear counting.	Have the linear counting algorithm pass all tests.
5	Mathematically analyse linear counting and compare against empirical results. Implement k-minimum values and HLL.	Have derivations match empirical results, and working KMV and HLL algorithms.
6	Analyse k-minimum values and HLL. Prepare write-up draft for progress check.	Code generates required graphics/figures.
7 (Stage II)	Implement Bloom Filter. Plan format and contents of poster.	Have preliminary results available for presentation.
8	Implement HLL++. Mathematically analyse Bloom Filter. Add rough available results and mathematical analysis to poster draft.	All algorithms tested and integrated.
9	Mathematically analyse HLL++. Design bulk of poster and finalise write-up.	Draft peer reviewed and extensions completed.
10 (Stage III)	Finalise poster and write-up with HLL++ results.	Submission ready.

1.5.2 Organisation and Challenge Resolution

Documents and programs will be shared via Github, and tasks will be tracked through GitHub Issues. Meetings and messages will be hosted on Discord/Teams with asynchronous messages whenever updates are made to keep everyone in the loop.

Each algorithm will have an implementation owner and a second member who reviews it and produces mathematical proofs as required. If an unresolvable discrepancy arises, another member of the group will arbitrate. Code changes will go through pull requests, approved by a third party. Meetings will be held weekly and findings will be updated in a shared draft of the project.

We will rotate practising the presentation of algorithms.

2 References

1 **Bar-Yossef, Z., Jayram, T. S., Kumar, R., Sivakumar, D. and Trevisan, L.** (2002) "Counting Distinct Elements in a Data Stream", *Randomization and Approximation Techniques in Computer Science*, volume 2483 of *Lecture Notes in Computer Science*, pages 1–10. DOI: 10.1007/3-540-45726-7_1

2 **Beyer, K. S., Haas, P. J., Reinwald, B., Sismanis, Y. and Gemulla, R.** (2007) "On synopses for distinct-value estimation under multiset operations", *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data*, pages 199–210. DOI: 10.1145/1247480.1247504

3 **Durand, M. and Flajolet, P.** (2003) "Loglog Counting of Large Cardinalities", *Algorithms - ESA 2003*, volume 2832 of *Lecture Notes in Computer Science*, pages 605–617. DOI: 10.1007/978-3-540-39658-1_55

4 **Flajolet, P. and Martin, G. N.** (1985) "Probabilistic counting algorithms for data base applications", *Journal of Computer and System Sciences*, volume 31, issue 2, pages 182–209. DOI: 10.1016/0022-0000(85)90041-8

5 **Flajolet, P., Fusy, É., Gandouet, O. and Meunier, F.** (2007) "HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm", *Proceedings of the AOFA 2007 (DMTCS Proceedings AH)*, pages 127–146. DOI: 10.1145/3534678.3539246

6 **Heule, S., Nunkesser, M. and Hall, A.** (2013) "HyperLogLog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm", *Proceedings of the 16th International Conference on Extending Database Technology*, pages 683–692. DOI: 10.1145/2452376.2452456

7 **QuestDB** 2025, *Cardinality estimation*, QuestDB, accessed October 4 2026, <https://questdb.com/glossary/cardinality-estimation/>

8 **Rahn, G.** 2016, *join-order-benchmark*, GitHub, accessed October 4 2026, <https://github.com/gregrahn/join-order-benchmark>

9 **UNSW** 2026, *Week 1 Workshop TeX Template*, UNSW Moodle, accessed September 18 2026, <https://moodle.telt.unsw.edu.au/pluginfile.php/15401209/mod_folder/content/0/Week\%201.tex>

10 **UNSW** 2025, *UNSW Harvard Referencing Guide*, UNSW Sydney, accessed October 4 2026, <https://www.unsw.edu.au/student/managing-your-studies/academic-skills-support/toolkit/referencing/harvard>

11 **Whang, K. Y., Vander-Zanden, B.T. and Taylor, H.M.** (1990) "A linear-time probabilistic counting algorithm for database applications", *ACM Transactions on Database Systems*, volume 15, issue 2, pages 208–229. DOI: 10.1145/78922.78925